

運用 Gemini Code Assist 大幅提升開發工作流程效率

來源：<https://codelabs.developers.google.com/codelabs/cloud-code-assist-sdlc?hl=zh-tw>

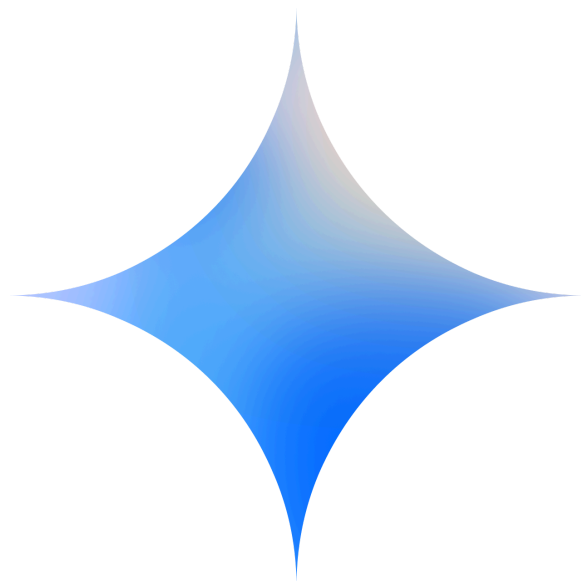
步驟數：16

在這個程式碼研究室中，您將瞭解 Gemini Code Assist 如何協助您在軟體開發生命週期 (SDLC) 的主要階段 (例如設計、建構與測試、測試與部署) 中提供協助。我們會設計並開發整個應用程式，然後部署至 Google Cloud。

目錄

1. 簡介
2. 設定 Gemini Code Assist
3. 設定 Firestore
4. 建立應用程式範本
5. 與 Gemini Code Assist 互動
6. API 設計
7. 產生應用程式
8. 程式碼重構
9. 與 Firestore 集合整合
10. 解釋程式碼
11. 生成網頁應用程式
12. 生成測試案例
13. 測試推動開發
14. 部署至 Google Cloud Run
15. (選用) 使用 Cloud Logging
16. 恭喜

1. 簡介



在本程式碼研究室中，您將瞭解 Gemini Code Assist 如何在軟體開發生命週期 (SDLC) 的主要階段 (例如設計、建構與測試，以及部署) 提供支援。我們會設計及開發整個應用程式，並部署到 Google Cloud。

我們將建構 API 和應用程式，以便在技術活動中搜尋各場次。每場講座都會有標題、摘要、時間長度、類別和一或多位講者。

執行步驟

- 從頭開始，根據 OpenAPI 規格設計、建構、測試及部署網頁應用程式

課程內容

- 如何使用 Gemini Code Assist 生成 OpenAPI 規格
- 如何使用 Gemini Code Assist 程式碼生成功能，為 OpenAPI 規格開發 Python Flask 應用程式
- 如何使用 Gemini Code Assist 為 Python Flask 應用程式生成網頁前端
- 如何使用 Gemini Code Assist，取得將應用程式部署至 Google Cloud Run 的相關協助
- 在建構及測試應用程式時，使用程式碼說明、測試案例生成等 Gemini Code Assist 功能

軟硬體需求

- Chrome 網路瀏覽器
- Gmail 帳戶
- 已啟用計費功能的 Cloud 專案
- 為雲端專案啟用 Gemini Code Assist

本實驗室適合各種程度的開發人員，包括初學者。雖然範例應用程式是以 Python 語言編寫，但您不需要熟悉 Python 程式設計，也能瞭解發生了什麼事。我們將著重於熟悉 Gemini Code Assist 的功能。

步驟 2 · 約 5 分鐘

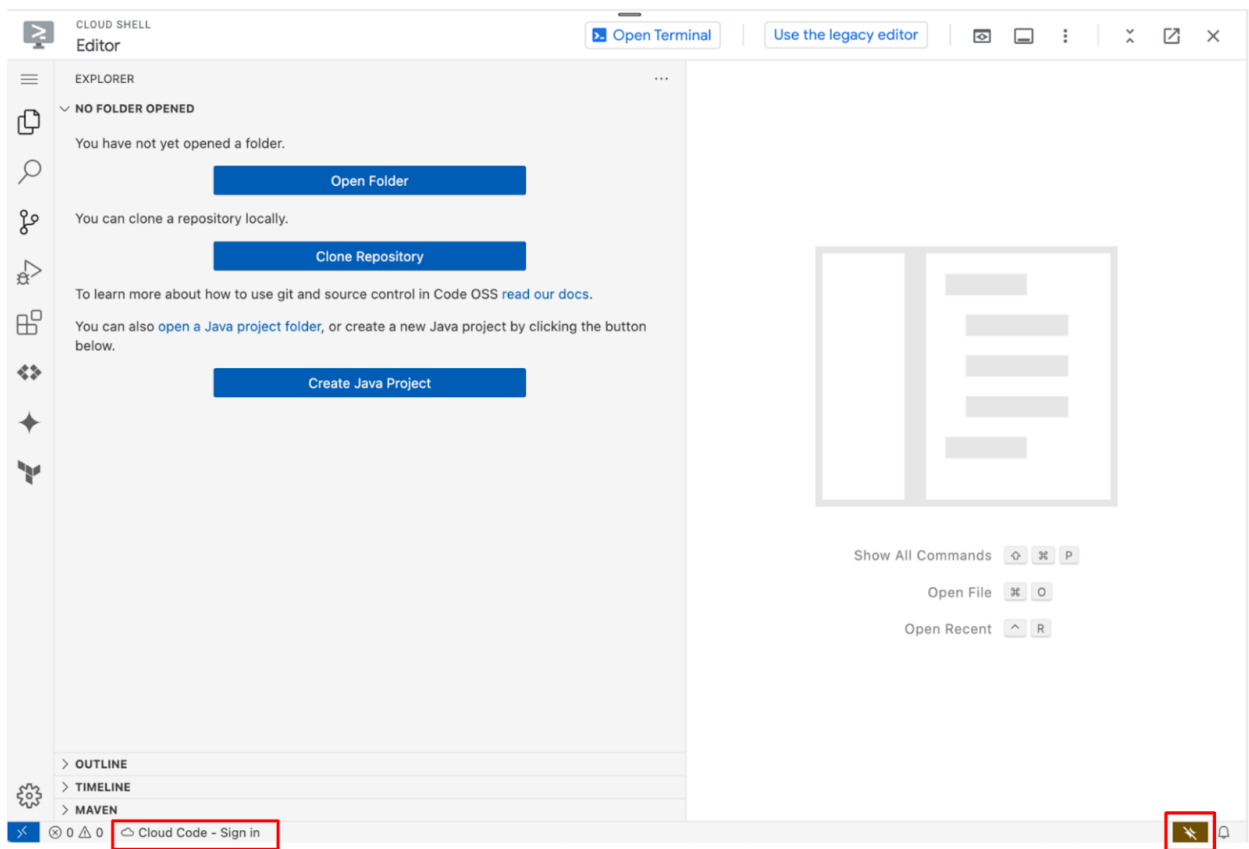
2. 設定 Gemini Code Assist

本節說明如何開始進行這個實驗室。

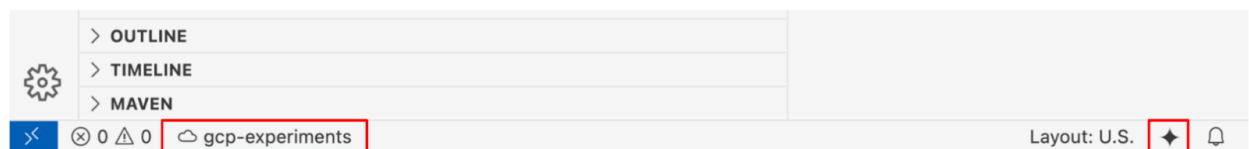
在 Cloud Shell IDE 中啟用 Gemini Code Assist

在接下來的程式碼研究室中，我們將使用 Cloud Shell IDE，這是以 [Code OSS](#) 為基礎的全代管開發環境。我們需要在 Cloud Shell IDE 中啟用及設定 Code Assist，步驟如下：

1. 前往 ide.cloud.google.com。IDE 可能需要一段時間才會顯示，請耐心等待並接受任何設定預設選項。如果看到設定 IDE 的操作說明，請繼續操作並採用預設設定。
2. 點選底部狀態列中的「Cloud Code - Sign in」按鈕，如下圖所示。依指示授權外掛程式。如果狀態列顯示「Cloud Code - no project」，請選取該項目，然後從專案清單中選取要使用的特定 Google Cloud 專案。



3. 如圖所示，按一下右下角的「Code Assist」按鈕，然後最後一次選取正確的 Google Cloud 專案。如果系統要求啟用 **Cloud AI Companion API**，請啟用並繼續操作。
4. 選取 Google Cloud 專案後，請確認狀態列的 Cloud Code 狀態訊息中已顯示該專案，且狀態列右側也已啟用 Code Assist，如下所示：



現在可以開始使用 Gemini Code Assist 了！

您也可以使用 [Visual Studio Code](#) 或 [JetBrains IntelliJ](#) 等本機 IDE 中使用 Gemini Code Assist。Google Cloud Workstations 也提供這項功能。本程式碼研究室著重於使用 Cloud Shell IDE，讓所有人都使用相同的環境，但您也可以嘗試慣用的設定，詳情請參閱[這個連結](#)。

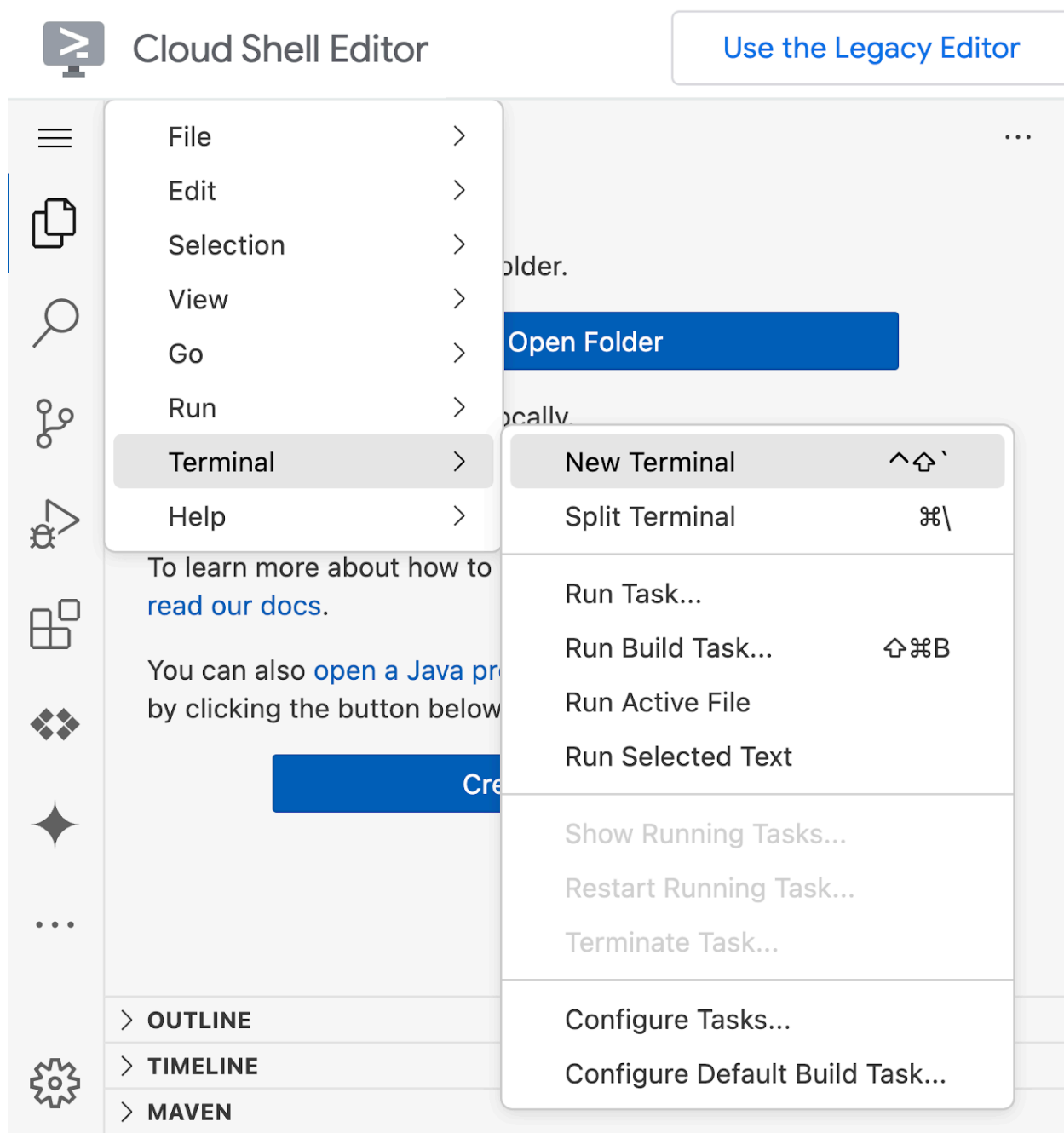
步驟 3 · 約 3 分鐘

3. 設定 Firestore

Cloud Firestore 是全代管的無伺服器文件資料庫，我們將用來做為應用程式資料的後端。Cloud Firestore 中的資料會以**文件集合**的形式建構。

我們需要在預設的 Firestore 資料庫中建立名為 **sessions** 的集合。這個集合會保存範例資料 (文件)，我們稍後會在應用程式中使用這些資料。

透過主選單在 Cloud Shell IDE 中開啟終端機，如下所示：



我們需要建立名為 **sessions** 的集合。這個資料夾會包含範例工作階段文件清單。每份文件都具有下列屬性：

1. **title**：字串
2. **categories**：字串陣列
3. **speakers**：字串陣列

4. **duration**：字串

5. **summary**：字串

讓我們將含有範例資料的檔案複製到您專案中的 bucket，然後透過 `gcloud firestore import` 指令匯入集合，藉此在集合中填入範例資料。

初始化 Firestore 資料庫

前往 Cloud 控制台的 [Firestore 頁面](#)。

如果專案中尚未初始化 Firestore 資料庫，請建立 **default** 資料庫。建立資料庫時，請使用下列值：

- Firestore 模式：**Native**
- 位置：選擇「位置類型」為 **Region**，然後選取適合應用程式的區域。請記下這個位置，因為下一個步驟會用到值區位置。
- 建立資料庫。

如果您已有 **default** 資料庫，請記下資料庫位置，因為您會在下一個步驟中需要用到值區位置。

Filter		
Database ID 	Mode	Location
(default)	 Native	asia-south1

現在請按照下列步驟建立 **sessions** 集合：

- 使用下列 `gsutil` 指令，在專案中建立 bucket。在下方指令中，將 `<PROJECT_ID>` 變數替換為您的 Google Cloud 專案 ID。將 `<BUCKET_LOCATION>` 替換為與預設 Firestore 資料庫地理區域對應的地區名稱 (如上一個步驟所述)，例如 US-WEST1、EUROPE-WEST1、ASIA-EAST1：

```
gsutil mb -l <BUCKET-LOCATION> gs://<PROJECT_ID>-my-bucket
```

- 建立值區後，我們需要將準備好的資料庫匯出內容複製到這個值區，才能匯入 Firebase 資料庫。使用下列指令：

```
gsutil cp -r gs://sessions-master-database-bucket/2024-03-26T09:28:15_95256  
gs://<PROJECT_ID>-my-bucket
```

現在我們已準備好要匯入的資料，可以進入最後一個步驟，將資料匯入我們建立的 Firebase 資料庫 (**default**)。

- 使用下列 `gcloud` 指令：

```
gcloud firestore import gs://<PROJECT_ID>-my-bucket/2024-03-26T09:28:15_95256
```

匯入作業需要幾秒鐘，完成後請前往 <https://console.cloud.google.com/firestore/databases>，選取 **default** 資料庫和 **sessions** 集合，驗證 Firestore 資料庫和集合，如下所示：

All databases > DATABASE (default) ⓘ

PANEL VIEW

QUERY BUILDER

/ > sessions > 20I3GhUU5h4qXYfDMPul ✎

(default)	sessions ⌵ ⋮	20I3GhUU5h4qXYfDMPul ⋮
+ START COLLECTION	+ ADD DOCUMENT	+ START COLLECTION
sessions >	20I3GhUU5h4qXYfDMPul >	+ ADD FIELD
	2i2oDKgL82yMBGNMUNF3 2mhLhmM4vM9L1vphKhwV 3yrLSpChD19cYurMh30X 5Ri75E7SV2E7nkvYG 111IK	categories 0: "AI" duration: "45 min" id: "DEV308" speakers (array) + 🗑 summary: "This session demonstrates ..." title: "Large language models in acti..."

請注意，如果您使用 Firebase，這就是您熟悉且喜愛的 Firestore。事實上，您可以使用 Firebase 控制台存取同一個資料庫執行個體！

這樣就完成了要在應用程式中使用的 Firestore 集合建立作業。

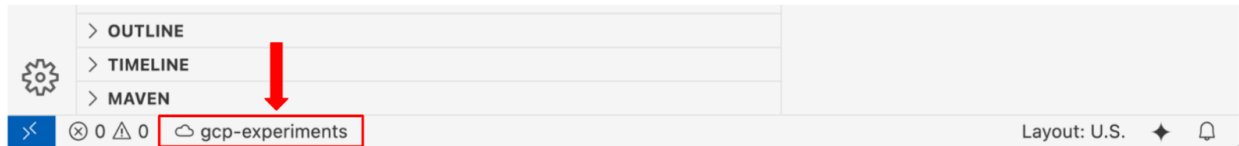
步驟 4 · 約 2 分鐘

4. 建立應用程式範本

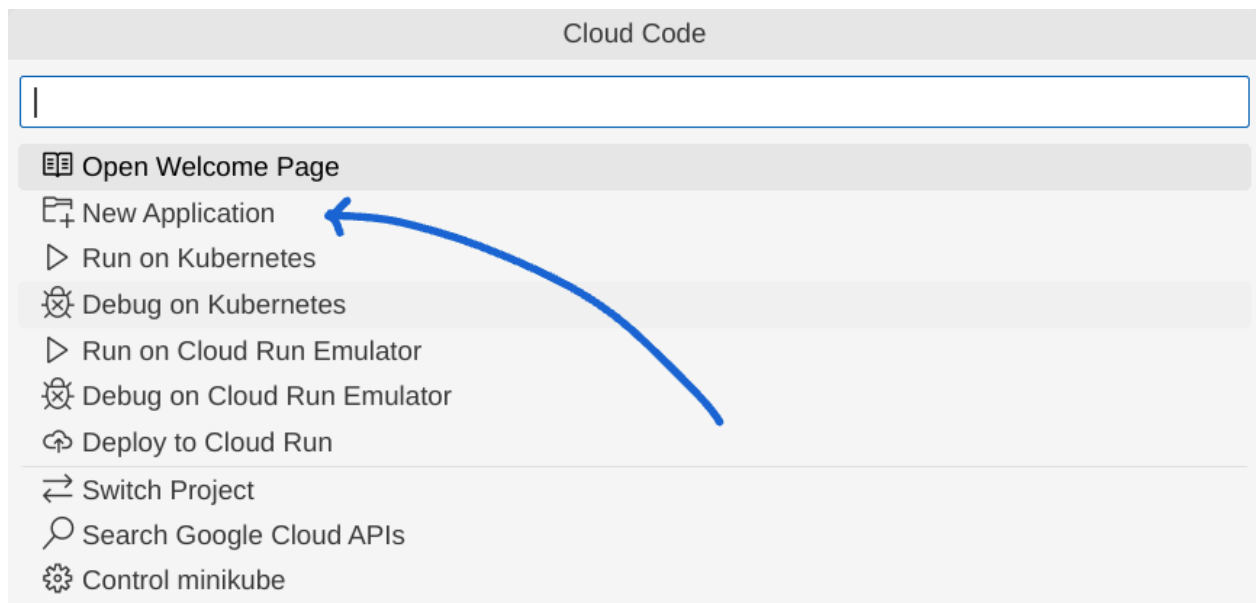
我們會建立範例應用程式 (Python Flask 應用程式)，在後續的程式碼研究室中使用。這個應用程式會搜尋技術研討會提供的所有課程。

步驟如下：

1. 按一下下方狀態列中的 **Google Cloud 專案名稱**。



2. 系統會顯示選項清單，按一下下方清單中的「New Application」。



3. 選取「Cloud Run application」（這會是應用程式的執行階段）。
4. 選取「Python (Flask): Cloud Run」應用程式範本。
5. 為應用程式命名，然後儲存到偏好位置。
6. 系統會顯示通知，確認您已建立應用程式，並開啟新視窗，載入應用程式，如下所示。系統會開啟 **README.md** 檔案。目前可以關閉該畫面。

Cloud Shell Editor

Use the Legacy Editor



EXPLORER

CODE-ASSIST-SDLC-DEMO

> .idea

> .readmes

> .vscode

> img

> static

> templates

 .dockerignore

 app.py

 Dockerfile

 Procfile

 README.md

 requirements.txt

 test_content.sh

> OUTLINE

> TIMELINE

Preview README.md

Cloud Run Hello World with Cloud Code

This "Hello World" sample demonstrates how to deploy a simple "Hello World" application to Cloud Run using the Cloud Code extension for Cloud Shell.

Table of Contents

- [Getting Started](#)
 - Run the app locally with the Cloud Run Emulator
 - Deploy to Cloud Run
- [Next steps](#)
- [Sign up for User Research](#)

Getting Started

Run the app locally with the Cloud Run Emulator

- Click on the Cloud Code status bar and select 'Run on Cloud Run Emulator'

 0 0 gcp-experiments

Layout: U.S.  

步驟 5 · 約 2 分鐘

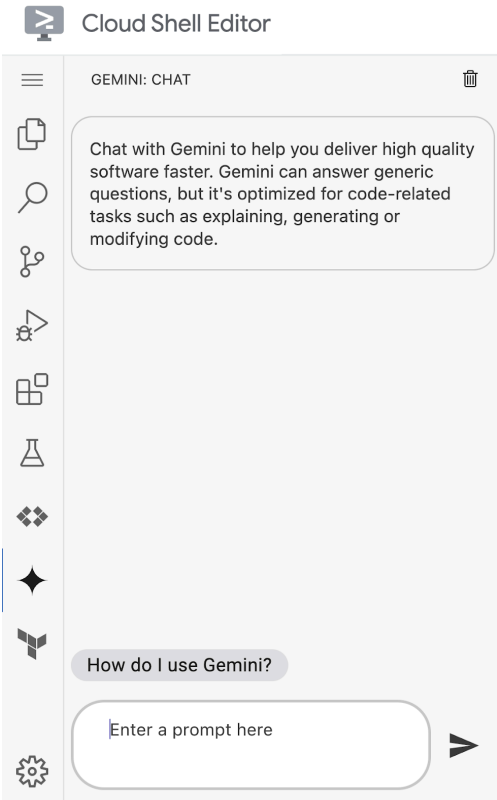
5. 與 Gemini Code Assist 互動

在本實驗室中，我們將使用 Cloud Shell IDE 內建的 Gemini Code Assist Chat，這是 VS Code 中 Cloud Code 擴充功能的一部分。如要開啟這項功能，請按一下左側導覽列中的「程式碼輔助」按鈕。在左側導覽工具列中尋找「程式碼輔助」圖示



，然後按一下。

這時 Cloud Shell IDE 會顯示 Code Assist 對話窗格，您可以在這裡與 Code Assist 對話。



請注意頂端的垃圾桶圖示，這是重設 Code Assist 聊天記錄環境的方法。另請注意，這項即時通訊互動會根據您在 IDE 中處理的檔案提供相關資訊。

步驟 6 · 約 3 分鐘

6. API 設計

第一步是在設計階段使用 Gemini Code Assist 輔助。在這個步驟中，我們將為要搜尋的實體 (活動中的技術課程) 生成 OpenAPI 規格。

輸入下列提示詞：

```
Generate an Open API spec that will allow me to retrieve all sessions, sessions by category, session by id. Each session has the following attributes: id, title, list of speakers, list of categories, summary and duration.
```

這應該會產生 OpenAPI 規格，可透過各種查詢參數搜尋工作階段。規格範例如下：

```
openapi: 3.0.0
info:
  title: Sessions API
  description: This API allows you to retrieve all sessions, sessions by category, and session
by id.
  version: 1.0.0
servers:
  - url: https://sessions.example.com
paths:
  /sessions:
    get:
      summary: Get all sessions
      operationId: getSession
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Session'
  /sessions/{id}:
    get:
      summary: Get session by id
      operationId: getSessionById
      parameters:
        - name: id
          in: path
          required: true
          description: The id of the session
          schema:
            type: string
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Session'
  /sessions/categories/{category}:
    get:
      summary: Get sessions by category
      operationId: getSessionByCategory
      parameters:
        - name: category
          in: path
          required: true
          description: The category of the sessions
          schema:
            type: string
      responses:
```

```

    '200':
      description: OK
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: '#/components/schemas/Session'
components:
  schemas:
    Session:
      type: object
      properties:
        id:
          type: string
          description: The id of the session
        title:
          type: string
          description: The title of the session
        speakers:
          type: array
          items:
            type: string
          description: The list of speakers for the session
        categories:
          type: array
          items:
            type: string
          description: The list of categories for the session
        summary:
          type: string
          description: The summary of the session
        duration:
          type: string
          description: The duration of the session

```

您會發現規格包含下列項目：

- 為「工作階段類型」定義的結構定義。
- 定義了數個 API 路徑：
- `/sessions`
- `/sessions/{id}`
- `/sessions/categories/{category}`

在頂層資料夾中建立名為 `sessionsapi.yaml` 的檔案，然後使用「插入目前檔案選項」(+ 按鈕) 從 Code Assist 即時通訊視窗複製內容，並在 Cloud Shell IDE 中開啟該檔案。

此時，您會發現 Gemini Code Assist 的一項有趣功能：[引文](#)。如果生成的程式碼直接完整引述其他來源的內容，例如現有的開放原始碼，系統就會向開發人員顯示這項資訊。並提供來源和授權，讓開發人員決定如何處理。

假設我們對生成的內容感到滿意，現在就可以使用這份規格文件，為其生成 Python Flask 應用程式。

步驟 7 · 約 5 分鐘

7. 產生應用程式

現在請 Code Assist 生成應用程式。開啟 `sessionsapi.yaml` 檔案，然後輸入下列提示詞。

```
Generate a Python Application using the Flask framework, based on the sessionsapi.yaml file.  
This application uses a local in memory list of sessions. Do not use any Flask extensions.
```

這應該會根據 OpenAPI 規格檔案中指定的功能和路徑，提供 Python Flask 應用程式的架構。

Gemini Code Assist 可能會使用特定 Flask 擴充功能 (例如 `flask_restful`) 生成程式碼。如果是這樣，您可以發出後續提示，明確指出「請勿使用 XYZ 擴充功能」等內容，系統就會提供使用 Flask 核心架構本身的程式碼。

提供的 Python Flask 應用程式程式碼應如下所示：

```

from flask import Flask, jsonify, request

app = Flask(__name__)

sessions = [
    {
        "id": "1",
        "title": "Session 1",
        "speakers": ["Speaker 1", "Speaker 2"],
        "categories": ["Category 1", "Category 2"],
        "summary": "This is a summary of session 1.",
        "duration": "1 hour",
    },
    {
        "id": "2",
        "title": "Session 2",
        "speakers": ["Speaker 3", "Speaker 4"],
        "categories": ["Category 3", "Category 4"],
        "summary": "This is a summary of session 2.",
        "duration": "1 hour 30 minutes",
    },
]

@app.route('/sessions', methods=['GET'])
def get_sessions():
    return jsonify(sessions)

@app.route('/sessions/<id>', methods=['GET'])
def get_session_by_id(id):
    session = next((session for session in sessions if session['id'] == id), None)
    if session is None:
        return jsonify({}), 404
    return jsonify(session)

@app.route('/sessions/categories/<category>', methods=['GET'])
def get_sessions_by_category(category):
    sessions_by_category = [session for session in sessions if category in
session['categories']]
    return jsonify(sessions_by_category)

if __name__ == '__main__':
    app.run()

```

您在上一個步驟中已產生 `app.py` 檔案。只要將檔案內容替換為 Code Assist 生成的程式碼，然後儲存檔案即可。

我們想變更 `app.run()` 行，使用通訊埠 8080、主機位址 0.0.0.0，並在本機執行期間以偵錯模式執行。方法如下：首先，請反白/選取該行：

```
app.run()
```

接著在 Code Assist Chat 介面中輸入提示：`Explain this.`

這時應該會顯示該行的詳細說明，如下所示：

The `app.run()` method in the provided Python Flask application is used to start the development server and run the application. It is typically called at the end of the script, after all the routes and other application logic have been defined.

When you run the application using `app.run()`, the following things happen:

1. The Flask development server is started. This server listens for incoming HTTP requests on a specified port (usually port 5000 by default).
2. When an HTTP request is received, the server dispatches the request to the appropriate route handler function based on the request's URL and HTTP method.
3. The route handler function processes the request and generates a response.
4. The server sends the response back to the client.

The `app.run()` method also includes several optional parameters that allow you to customize the behavior of the development server, such as:

- **host**: The hostname or IP address to listen on.
- **port**: The port number to listen on.
- **debug**: A boolean value that enables or disables debug mode. In debug mode, the server will automatically restart when you make changes to your code.
- **threaded**: A boolean value that enables or disables multithreading. In multithreaded mode, the server can handle multiple requests concurrently.

By default, the `app.run()` method will start the development server on localhost (127.0.0.1) and listen on port 5000. You can change these settings by passing the appropriate parameters to the `app.run()` method.

For example, to start the server on port 8000, you would use the following code:

```
app.run(port=8000)
```

To start the server in debug mode, you would use the following code:

```
app.run(debug=True)
```

To start the server in multithreaded mode, you would use the following code:

```
app.run(threaded=True)
```

Once the server is running, you can access your application by visiting the specified URL in a web browser. For example, if you started the server on localhost:8000, you would visit the following URL in your browser:

```
http://localhost:8000
```

Rate this answer:  

現在請使用下列提示：

update the code to run the application on port 8080, host address 0.0.0.0, and in debug mode

生成的建議程式碼應如下所示：

```
app.run(host='0.0.0.0', port=8080, debug=True)
```

請記得使用這個程式碼片段更新 `app.py` 檔案。

在本機執行應用程式

現在請在本機執行應用程式，根據我們一開始的設定驗證應用程式需求。

第一步是建立 Python 虛擬環境，並在虛擬環境中安裝 **requirements.txt** 中的 Python 套件依附元件。如要執行這項操作，請前往 Cloud Shell IDE 的指令面板 (Ctrl+Shift+P)，然後輸入「Create Python environment」。請按照接下來的幾個步驟，選取「虛擬環境 (venv)」、「Python 3.x 解譯器」和「requirements.txt」檔案。

建立環境後，啟動新的終端機視窗 (Ctrl+Shift+`) 並輸入下列指令：

```
python app.py
```

執行範例如下所示：

```
(.venv) romin@cloudshell: $ python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a
production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://10.88.0.3:8080
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 132-247-368
```

您現在可以透過下列網址預覽 API。我們假設您的開發伺服器正在通訊埠 8080 上執行。如果不是，請改用適當的通訊埠號碼。

- `https://<host-name>:8080/sessions`
- `https://<host-name>:8080/sessions/{id}`
- `https://<host-name>:8080/sessions/categories/{category}`

請按照下列步驟操作，確保您能使用這些網址擷取 **app.py** 檔案中的 JSON 資料：

開啟新的終端機視窗，然後嘗試執行下列任一指令：

```
curl -X GET http://127.0.0.1:8080/sessions
curl -X GET http://127.0.0.1:8080/sessions/<ID>
curl -X GET http://127.0.0.1:8080/sessions/categories/<CATEGORY_NAME>
```

請注意，如果類別名稱需要空格，可以使用 %20 做為值。

步驟 8 · 約 3 分鐘

8. 程式碼重構

我們可能想將硬式編碼的範例 JSON 資料從 `app.py` 中分離/擷取到另一個模組，以便清楚區分程式碼和資料。我們來看看！

保持 `app.py` 檔案開啟，然後輸入下列提示：

Can I improve this code and separate out the sessions data from this app.py file?

系統隨即會提供相關建議。以下是我們取得的建議範例，您應該會看到類似的內容：

Yes, you can improve the code and separate out the sessions data from the `app.py` file by creating a separate module for the sessions data. Here's an example of how you can do it:

Create a new file called `sessions.py` in the same directory as `app.py`:

```
sessions = [
    {
        "id": "1",
        "title": "Session 1",
        "speakers": ["Speaker 1", "Speaker 2"],
        "categories": ["Category 1", "Category 2"],
        "summary": "This is a summary of session 1.",
        "duration": "1 hour",
    },
    {
        "id": "2",
        "title": "Session 2",
        "speakers": ["Speaker 3", "Speaker 4"],
        "categories": ["Category 3", "Category 4"],
        "summary": "This is a summary of session 2.",
        "duration": "1 hour 30 minutes",
    },
]
```

Import the `sessions` module in the `app.py` file:

```
from sessions import sessions
```

Update the `app.py` file to use the `sessions` module:

```
from flask import Flask, jsonify, request
from sessions import sessions

app = Flask(__name__)

@app.route('/sessions', methods=['GET'])
def get_sessions():
    return jsonify(sessions)

@app.route('/sessions/<id>', methods=['GET'])
def get_session_by_id(id):
    session = next((session for session in sessions if session['id'] == id), None)
    if session is None:
        return jsonify({}), 404
    return jsonify(session)

@app.route('/sessions/categories/<category>', methods=['GET'])
def get_sessions_by_category(category):
    sessions_by_category = [session for session in sessions if category in session['categories']]
    return jsonify(sessions_by_category)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

讓我們按照建議，將資料分成 `sessions.py` 檔案。

建立名為 `sessions.py` 的新檔案

，其內容為 JSON 清單，如下所示：

```
sessions = [
    {
        "id": "1",
        "title": "Session 1",
        "speakers": ["Speaker 1", "Speaker 2"],
        "categories": ["Category 1", "Category 2"],
        "summary": "This is a summary of session 1.",
        "duration": "1 hour",
    },
    {
        "id": "2",
        "title": "Session 2",
        "speakers": ["Speaker 3", "Speaker 4"],
        "categories": ["Category 3", "Category 4"],
        "summary": "This is a summary of session 2.",
        "duration": "1 hour 30 minutes",
    },
]
```

`app.py` 檔案現在已大幅簡化，如下所示：

```
from flask import Flask, jsonify, request
from sessions import sessions

app = Flask(__name__)

@app.route('/sessions', methods=['GET'])
def get_sessions():
    return jsonify(sessions.sessions)

@app.route('/sessions/<id>', methods=['GET'])
def get_session_by_id(id):
    session = next((session for session in sessions.sessions if session['id'] == id), None)
    if session is None:
        return jsonify({}), 404
    return jsonify(session)

@app.route('/sessions/categories/<category>', methods=['GET'])
def get_sessions_by_category(category):
    sessions_by_category = [session for session in sessions.sessions if category in session['categories']]
    return jsonify(sessions_by_category)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

請確認您仍可在本機執行應用程式，並透過這些重構變更運用 API。Python 開發伺服器可能仍在執行中，因此您只需要回想上一步的 `curl` 指令。

步驟 9 · 約 4 分鐘

9. 與 Firestore 集合整合

下一步是捨棄我們擁有的本機記憶體內 JSON 工作階段清單，並將應用程式連線至 Firestore 資料庫中的 **sessions** 集合（我們在本程式碼研究室一開始建立的集合）。

保持 **sessions.py** 檔案開啟，然後輸入下列提示：

```
Can you further refactor the sessions.py code to read from a Firestore database that has a collection named sessions. The collection has the same attributes as the session object that we have defined. Use the Python module google-cloud-firestore.
```

我們收到以下建議，要從 Firestore 集合讀取所有工作階段：

```
import google.cloud.firestore

# Create a Firestore client
db = google.cloud.firestore.Client()

# Get the sessions collection
sessions_collection = db.collection("sessions")

# Create a list of sessions
sessions = []

# Iterate over the documents and add them to the list
for doc in sessions_collection.stream():
    session = doc.to_dict()
    session["id"] = doc.id
    sessions.append(session)
```

請在 **sessions.py** 中加入程式碼。

如果您在本機執行 Flask 開發伺服器，應用程式可能會關閉，並顯示找不到 Python 模組的錯誤訊息。

舉例來說，您可以詢問 Code Assist 應將哪個 Python 模組新增至 **requirements.txt** 檔案，如下所示：

```
Which Python package needs to be installed to make the firestore code work?
```

這會提供 Python 模組的名稱（例如 **google-cloud-firestore**）。將該名稱新增至 **requirements.txt** 檔案。

您需要使用新加入的模組（**google-cloud-firestore**）重新建立 Python 環境。如要這麼做，請在現有的終端機視窗中輸入下列指令：

```
pip install -r requirements.txt
```

再次執行應用程式（使用 **python app.py** 重新啟動），然後前往 **/sessions** 網址。您現在應該會取得我們新增至 **sessions** 集合的範例文件。

```
[
  {
    "categories": [
      "AI"
    ],
    "duration": "45",
    "id": "0Drs49eXX4MEbC8kiLJa",
    "speakers": [
      "Avanti Sane",
      "Vishy Tirumalashetty"
    ],
    "summary": "AI-generated videos are a game-changer for enterprises. Whether it is advertising, marketing, e-commerce, or entertainment, AI-generated videos can help you create more appealing, engaging, and personalized customer experiences and forever change how you interact with your consumers. Text-to-video and image-to-video capabilities can help create more compelling stories, immersive visuals, and amazing content. Attend this session and learn how Vertex AI can help you create the next generation of customer delight.",
    "title": "Deliver the next generation of customer experiences with AI generated video"
  },
  {
    "categories": [],
    "duration": "45",
    "id": "0NxGf10iMBYZSmQhx53e",
    "speakers": [
      "James Manos"
    ],
    "summary": "Learn how Landis+Gyr, a leading manufacturer and operator of smart meters, modernized their legacy systems and moved to a cloud-first architecture using infrastructure as code. The session reflects on how Landis+Gyr embarked on their journey from manually provisioning hardware and compute environments for individual smart metering customer wins to harnessing the power of cloud and infrastructure as code to support a large-scale modernization and replatforming project of Landis+Gyr's backend for smart meter data and data-driven services.",
    "title": "Unleash the power of the cloud in energy"
  },
  {
    "categories": [
      "AI"
    ],
    "duration": "45",
    "id": "0WbnLQw0GDYQscgF17M0",
    "speakers": [
      "Adam Thorsteinson",
      "Saif Abid",
      "Francois Nicholas"
    ],
    "summary": "Conversational large language model (LLM) agents have the potential to revolutionize the online customer journey, but they carry substantial risks. How can we evaluate the nuance and effectiveness of an agent's conversational outputs when there isn't a ground truth for comparison, and without using hundreds of hours of"
  }
]
```

歡迎查詢其他 URI，擷取特定工作階段，或如先前步驟所述，擷取特定類別的所有工作階段。

步驟 10 · 約 1 分鐘

10. 解釋程式碼

現在是使用 Gemini Code Assist 的 **"Explain this"** 功能，深入瞭解程式碼的好時機。您可以隨意進入任何檔案，或選取特定程式碼片段，然後使用下列提示詞要求 Code Assist 執行動作：**Explain this**。

請前往 **sessions.py** 檔案，醒目顯示 Firestore 專屬程式碼，並取得相關程式碼說明。您也可以專案中的其他檔案上使用這項功能，不限於 Python 程式碼。

11. 生成網頁應用程式

我們已產生 API 並與即時 Firestore 集合整合，現在要為應用程式產生網頁式前端。目前網頁前端會盡量簡化功能，也就是能夠搜尋屬於特定類別的會話。請注意，我們有該 API 路徑 (即 `/sessions/categories/{category}`)，因此網路應用程式應叫用該路徑並擷取結果。

我們馬上進入正題！對 Code Assist 輸入下列提示：

```
Generate a web application that allows me to search for sessions by category and uses the Flask application that we created. Please use basic HTML, CSS and JS. Embed all the Javascript and CSS code into a single HTML file only.
```

這會產生網頁應用程式 HTML，並在其中嵌入 JavaScript 和 CSS。系統也會要求您在 `app.py` 檔案中新增路徑，讓造訪根網址或基準網址的使用者都能看到首頁。如果沒有，請詢問或使用下列程式碼片段：

```
@app.route('/')
def index():
    return render_template('index.html')
```

您可以將這個檔案儲存為 `index.html`，但可能有疑問，這個檔案應該儲存在哪裡 (也就是哪個資料夾)。我們可以向 Code Assist 提出後續問題。

```
Given that I am using the flask framework, where should I put the index.html file?
```

當中應清楚說明該應用程式使用 `render_template` 架構，因此 `index.html` 檔案必須放在 `templates` 資料夾中。在本程式碼研究室一開始，我們就根據 Flask 範本產生了應用程式，因此您會發現這個資料夾。因此，系統會產生現有的 `index.html` 檔案，您只需將其內容替換成這裡產生的新內容即可。Code Assist 也會提及在 `app.py` 檔案中匯入 `render_template`。

請將 Web 應用程式程式碼儲存到 `index.html` 檔案，並記得將該檔案放在 `templates` 資料夾中。

在本機執行應用程式

現在請在本機執行應用程式，根據我們一開始的設定驗證應用程式需求。

確認本機 Flask 伺服器仍在執行中，且未顯示任何錯誤。如有錯誤，請修正並啟動伺服器。

啟動並執行後，請前往應用程式的主網址。如下所示，畫面應會顯示所提供的 `index.html` 檔案：

Sessions App

Category:

提供資料中的其中一個類別做為輸入內容 (例如 `AI`)，然後按一下 `Search` 按鈕。這時應該會顯示標記為 `AI` 類別的會話。

Sessions App

Category:

[Large language models in action: Transforming ideas into cloud architecture blueprints](#)
[Accelerate software delivery with Gemini and Code Transformations](#)
[Secure your generative AI developer workflow with Snyk and Google Cloud](#)
[Go from large language model to market faster with Ray, Hugging Face, and LangChain](#)
[How to leave the office two hours early with Gemini in the Cloud Console](#)
[Supercharge your development workflow with Duet AI](#)
[Introducing Google Cloud's open source, cloud-optimized framework for building mobile and web applications with generative AI](#)
[Get developer assistance customized to your organization code with Duet AI](#)
[How Locket Labs scales their workloads: Leveraging generative AI apps with Firebase and Firestore](#)
[Five ways AI-assisted API automation can supercharge your platform engineering](#)
[Define, don't draw: Leverage AI to summarize your software architecture](#)
[Accelerate your AI with Serverless](#)
[Build full stack applications using Firebase and Google Cloud.](#)
[Run large-scale AI training and inference for Llama 2 on Google Cloud Accelerators](#)
[Build Gen AI applications with Google Databases and Cloud Runtimes](#)
[Gemini codes live for you with Cloud Code](#)
[Dev to DevAI: The future of app development with analytics and gen AI](#)
[What's next for Google AI and Vertex AI software development kits and the Gemini API](#)
[Gemini: Lessons from building 30-plus generative AI-powered developer tools](#)
[How innovators are using generative AI to evaluate large language model chatbots at scale](#)
[When AI meets data: Stack Overflow and Google Cloud Duet AI](#)
[Roll up your sleeves: Craft real-world generative AI Java in Cloud Run](#)

選用練習：顯示每個講座的其他中繼資料，例如時間長度、類別、講者和摘要。

12. 生成測試案例

我們已生成 Sessions API，現在可以使用 Gemini Code Assist，為不同的 API 路徑生成單元測試案例。

保持 `app.py` 檔案開啟，然後輸入下列提示：

```
Generate unit test cases for app routes. Use unittest module.
```

我們收到以下回覆：

```
import unittest
from app import app

class FlaskAppTestCase(unittest.TestCase):

    def test_index(self):
        tester = app.test_client()
        response = tester.get('/')
        self.assertEqual(response.status_code, 200)

    def test_get_sessions(self):
        tester = app.test_client()
        response = tester.get('/sessions')
        self.assertEqual(response.status_code, 200)

    def test_get_session_by_id(self):
        tester = app.test_client()
        response = tester.get('/sessions/1')
        self.assertEqual(response.status_code, 200)

    def test_get_sessions_by_category(self):
        tester = app.test_client()
        response = tester.get('/sessions/categories/category1')
        self.assertEqual(response.status_code, 200)

if __name__ == '__main__':
    unittest.main()
```

建立名為 `tests.py` 的檔案，並在其中加入上述程式碼。

測試案例生成注意事項

您看到的程式碼清單可能與上述不同，這可能會導致執行測試案例時發生一些問題。舉例來說，我們發現在某些執行作業中，下列重要程式碼片段遭到遺漏：

```
from app import app
```

上述程式碼用於匯入現有的 Flask 應用程式，我們將針對該應用程式叫用測試案例。

```
if __name__ == '__main__':
```

```
`unittest.main()`
```

執行測試案例時需要上述程式碼。

建議您查看每個測試案例，檢查生成的程式碼中的 `assertEqual` 和其他條件，確保程式碼能正常運作。由於資料位於 Firestore 集合中的外部，因此可能無法存取，並可能使用一些虛擬資料，導致測試失敗。因此請視情況修改測試案例，或註解掉您可能不需要立即執行的測試案例。

為進行示範，我們使用下列指令執行測試案例 (請務必執行本機開發伺服器，因為系統會呼叫本機 API 端點)：

```
python tests.py
```

我們得到以下摘要結果：

```
Ran 4 tests in 0.274s
```

```
FAILED (failures=2)
```

這確實是正確的，因為第 3 次測試的工作階段 ID 不正確，且沒有名為 `category1` 的類別

。

因此請據此調整測試案例，然後進行測試。

13. 測試推動開發

現在，我們將按照測試驅動開發 (TDD) 方法，在工作階段 API 中新增搜尋方法。TDD 方法是先編寫測試案例，然後因為缺少實作內容而導致測試失敗，最後使用 Gemini Code Assist 生成缺少的實作內容，讓測試通過。

前往 `tests.py` 檔案 (假設您已修正 `tests.py` 檔案，讓所有測試都通過)。向 Code Assist 提出下列提示：

```
Generate a new test case to search for sessions by speaker
```

這讓我們獲得下列測試案例實作項目，並適時插入 `tests.py` 檔案。

```
def test_get_sessions_by_speaker(self):
    tester = app.test_client()
    response = tester.get('/sessions/speakers/speaker1')
    self.assertEqual(response.status_code, 200)
    self.assertEqual(response.json, [sessions.sessions[0], sessions.sessions[1]])
```

執行測試後，您應該會看到下列錯誤訊息：

```
$ python tests.py
.F.
=====
FAIL: test_get_sessions_by_speaker (__main__.FlaskAppTestCase)
-----
Traceback (most recent call last):
  File "/home/romin/hello-world-5/tests.py", line 21, in test_get_sessions_by_speaker
    self.assertEqual(response.status_code, 200)
AssertionError: 404 != 200

-----
Ran 3 tests in 0.010s

FAILED (failures=1)
```

這是因為測試案例已叫用下列路徑 (`/sessions/speakers/`)，但 `app.py` 中沒有該路徑的實作項目。

請 Code Assist 提供實作方式。前往 `app.py` 檔案，並向 Code Assist 輸入下列提示：

```
Add a new route to search for sessions by a specific speaker
```

我們採用 Code Assist 建議的下列實作項目，並新增至 `app.py` 檔案：

```
@app.route('/sessions/speakers/<speaker>', methods=['GET'])
def get_sessions_by_speaker(speaker):
    sessions_by_speaker = [session for session in sessions.sessions if speaker in
session['speakers']]
    return jsonify(sessions_by_speaker)
```

請重新查看 `tests.py` 檔案，我們已修改測試案例，以便快速檢查：

```
def test_get_sessions_by_speaker(self):
    tester = app.test_client()
    response = tester.get('/sessions/speakers/Romin Irani')
    self.assertEqual(response.status_code, 200)
    self.assertEqual(len(response.json), 1)
```

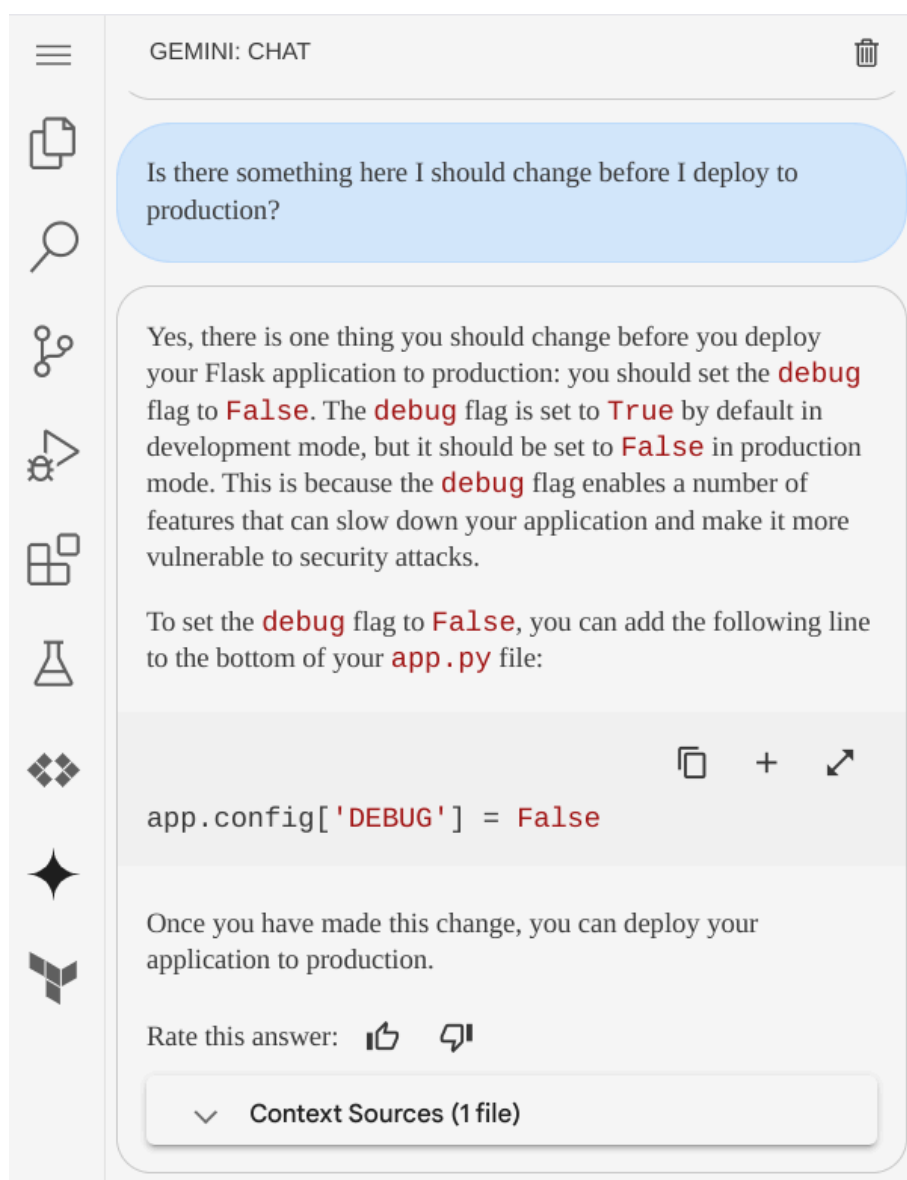
測試執行正常。我們將這項練習留給您，請查看產生的測試案例，並根據 Firestore 中的資料稍微調整，然後在 Python 單元測試案例中加入適當的 `assert*` 方法。

14. 部署至 Google Cloud Run

我們對開發品質感到滿意，最後一個步驟是將這個應用程式部署至 Google Cloud Run。但為了確保萬無一失，我們應該詢問 Code Assist 是否有任何遺漏。開啟 `app.py`，然後提交下列提示：

Is there something here I should change before I deploy to production?

幸好你問了，因為我們忘了將偵錯標記設為關閉：



如圖所示，關閉偵錯功能，然後要求 Gemini Code Assist 提供 `gcloud` 指令，直接從來源將應用程式部署至 Cloud Run (不必先建構容器)。

輸入下列提示詞：

I would like to deploy the application to Cloud Run directly from source. What is the gcloud command to do that?

請嘗試上述提示的幾種變體。我們也嘗試了以下做法：

I would like to deploy this application to Cloud Run. I don't want to build a container image locally but deploy directly from source to Cloud Run. What is the gcloud command for that?

理想情況下，您應該會收到下列 **gcloud** 指令：

```
gcloud run deploy sessions --source .
```

你可能也會獲得：

```
gcloud run deploy <service-name> --source . \
--platform managed \
--allow-unauthenticated
```

從應用程式的根資料夾執行上述指令。系統要求 **region** 時，請選取 **us-central1**；系統要求允許 **unauthenticated invocations** 時，請選擇 **y**。系統也可能會要求您啟用 Artifact Registry、Cloud Build 和 Cloud Run 等 Google Cloud API，並授予建立 Artifact Registry 存放區的權限，請繼續操作並授予權限。

部署程序大約需要 2 分鐘才能完成，請耐心等待。

成功部署後，您會看到 Cloud Run 服務網址。前往該公開網址，您應該會看到已部署且成功執行的相同網頁應用程式。

由於 Cloud Run 應用程式會以自己的服務身分執行，因此請務必確保與 Cloud Run 服務相關聯的服務帳戶具備存取 Firestore 資料庫的必要權限。請務必確認 Cloud Run 服務帳戶是否具備 **roles/datastore.user** 角色。

Sessions App

Category:

[How Snapchat secures its services on multi-tenant Google Kubernetes Engine](#)

[Retail reimagined: Where AI, serverless computing, data, and APIs meet](#)

[Secure cloud developer workplaces: How Xebia empowers clients with Google Cloud Workstations](#)

[Build an internal developer portal with Backstage to accelerate Google Cloud adoption](#)

[Modernize consumer devices with Google Cloud: Deep dive into MosquitoMax's story](#)

[How HSBC built a "migration factory" to replatform Websphere at scale](#)

[Simplifying app troubleshooting with Google Cloud](#)

[Why Google Kubernetes Engine uniquely supports the reliability-first approach](#)

[Ford Motor Co.'s acceleration to Google Cloud fueled by Cloud Run](#)

[Reimagining customer experiences through cloud runtimes](#)

[How to deploy all the JavaScript frameworks to Cloud Run](#)

[How ANZ delivers reliable applications on Google Cloud using archetype-based platforms](#)

[Build an integrated DevSecOps solution with GitLab and Google Cloud](#)

[How Google Kubernetes Engine autopilot leveraged Ubie's re-architecting of its microservices](#)

[Cloud Run: what's new](#)

[Java on Google Cloud: The enterprise, the serverless, and the native](#)

[Mainframe modernization: Assess, refactor, and rewrite using generative AI](#)

恭喜，做得好！

15. (選用) 使用 Cloud Logging

我們可以在應用程式中導入記錄功能，讓應用程式記錄集中在其中一項 Google Cloud 服務 (Cloud Logging)。接著，我們可以使用 Observability Gemini 功能，瞭解記錄項目。

為此，我們必須先使用 Google Cloud 的現有 Python Cloud Logging 程式庫，並根據記錄 / 嚴重性等級，使用該程式庫記錄資訊、警告或錯誤訊息。

我們先試著向 Code Assist 提出這個問題。請嘗試使用下列提示詞：

```
How do I use the google-cloud-logging package in Python?
```

您應該會收到類似下方的回應，提供相關資訊：

To use the **google-cloud-logging** package in Python, you can follow these steps:

1. Install the package using pip:

```
pip install google-cloud-logging
```

2. Import the package into your Python script:

```
from google.cloud import logging
```

3. Create a logger object:

```
logger = logging.Client().logger('my-log')
```

4. Use the logger object to log messages:

```
logger.log_text('Hello, world!')
```

You can also use the logger object to log structured data, such as JSON objects:

```
logger.log_struct({'name': 'John Doe', 'age': 30})
```

The **google-cloud-logging** package provides a number of other features, such as the ability to filter and search log entries, and to create and manage log sinks.

我們來為依類別搜尋工作階段的函式新增記錄陳述式。

首先，請將 `google-cloud-logging` Python 套件新增至 `requirements.txt` 檔案。

接下來是程式碼片段，說明我們如何整合程式碼來實作記錄功能：

```
...
from google.cloud import logging
...
app = Flask(__name__)

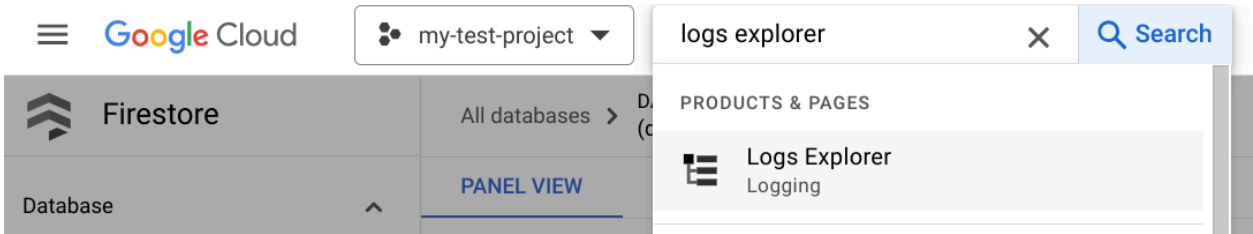
# Create a logger
logger = logging.Client().logger('my-log')

@app.route('/sessions/categories/<category>', methods=[ 'GET' ])
def get_sessions_by_category(category):
    logger.log_text(f"Fetching sessions with category {category}")
    sessions_by_category = [session for session in sessions.sessions if category in
session['categories']]
    logger.log_text(f'Found {len(sessions_by_category)} sessions with category {category}')
    return jsonify(sessions_by_category)

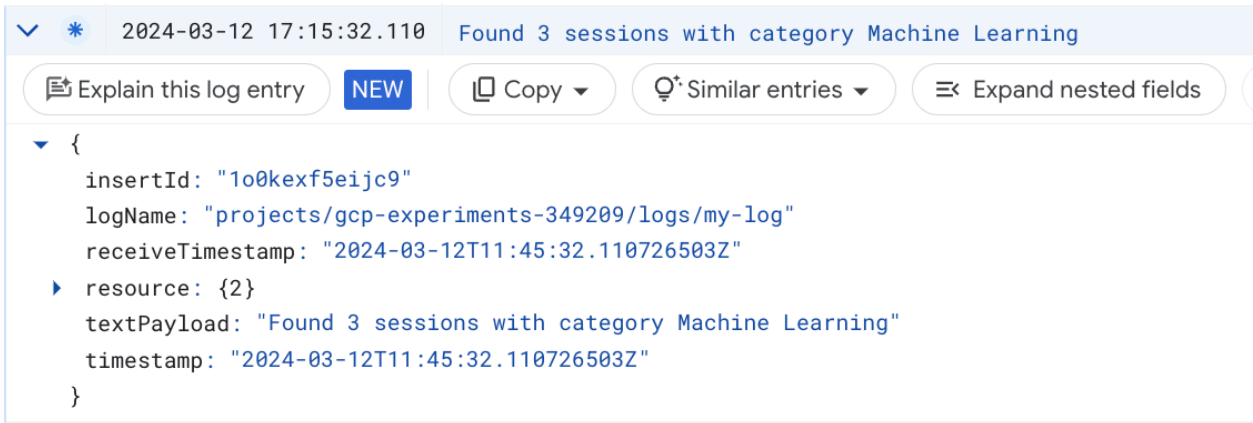
# # Other App Routes
```

使用與上一節相同的指令，再次將服務部署至 Cloud Run，部署完成後，對 `/sessions/categories/<category>` 端點執行幾次呼叫。

前往 `Cloud Console` → `Logs Explorer`



...您應該可以篩選這些記錄陳述式，如下所示：



您可以點選任一記錄陳述式，展開該陳述式，然後點選 `Explain this log entry`，系統就會使用 Gemini 說明記錄項目。請注意，如果您尚未啟用 Gemini for Google Cloud，系統會要求您啟用 Cloud AI Companion API。請按照指示操作。

回應範例如下所示：

Explain this log entry in 3 to 5 bullet points.

▼ Additional prompt details

- The log entry was generated by a Cloud Run service named `sessions` in the `us-central1` region.
- The project ID associated with the log entry is `gcp-experiments-349209` .
- The log entry was generated by revision `sessions-00001-4zv` of the `sessions` service.
- The log entry has a `textPayload` field with the value `Found 3 sessions with category Machine Learning` .
- The log entry was received by Cloud Logging at `2024-03-12T11:45:32.110726503Z` .

Rate this answer:



步驟 16 · 約 0 分鐘

16. 恭喜

恭喜！您已從頭開始建構應用程式，並在 SDLC 的多個方面 (包括設計、建構、測試和部署) 使用 Gemini Code Assist。

後續步驟

查看一些程式碼研究室...

- [開發人員專用 Duet AI 導覽](#)
- [在軟體開發生命週期中使用 Duet AI](#)
- [使用開發人員適用的 Duet AI 打造時尚風格](#)

參考文件

- [Gemini Code Assist 說明文件](#)
 - [為 Google Cloud 中的 Gemini 撰寫更有效的提示詞](#)
-